WikiSkill：将智能体经验编译为支持技能演化的持久化知识

Liyan Tang[1], **Cyrus Rashtchian**[1], **Chun-Sung Ferng**[1], **Andrew Tomkins**[1], **Da-Cheng Juan**[1]、**Tu Vu**[1,2]
[1]Google 研究院, [2] 弗吉尼亚理工大学

智能体技能将专门知识与工作流封装为可复用资源，从而扩展 AI 智能体的能力。近期工作从智能体经验中自动发现此类技能，使智能体能够通过交互逐步适应。然而，指导技能开发的洞见通常分散在优化历史之中，限制了这些洞见在不同迭代间的系统化复用。我们提出 WikiSkill，这是一个让智能体技能与持久化知识库（Wiki）协同演化的框架。从高层来看，WikiSkill 将原始执行经验、累积知识和可执行技能彼此分离，同时持续将经验整合进 Wiki，使后续技能更新能够以此为基础。在多种基准和模型上，WikiSkill 始终优于当前最先进的技能演化方法，并在大多数模型–基准组合中优于无技能基线。我们发现，技能演化与模型规模扩展相互补充：较大模型通常能从演化所得技能中获益更多，而配备技能的较小模型可以胜过规模大得多但不配备技能的模型。我们还发现，演化所得技能能够在不同模型及模型家族之间有效迁移，而且由其他模型演化得到的技能可能优于模型自行演化的技能。最后，消融研究证实，Wiki 中的持久化知识累积对于有效的技能演化至关重要。这些结果表明，系统性地累积和提炼智能体经验有助于开发可复用、可迁移的技能。

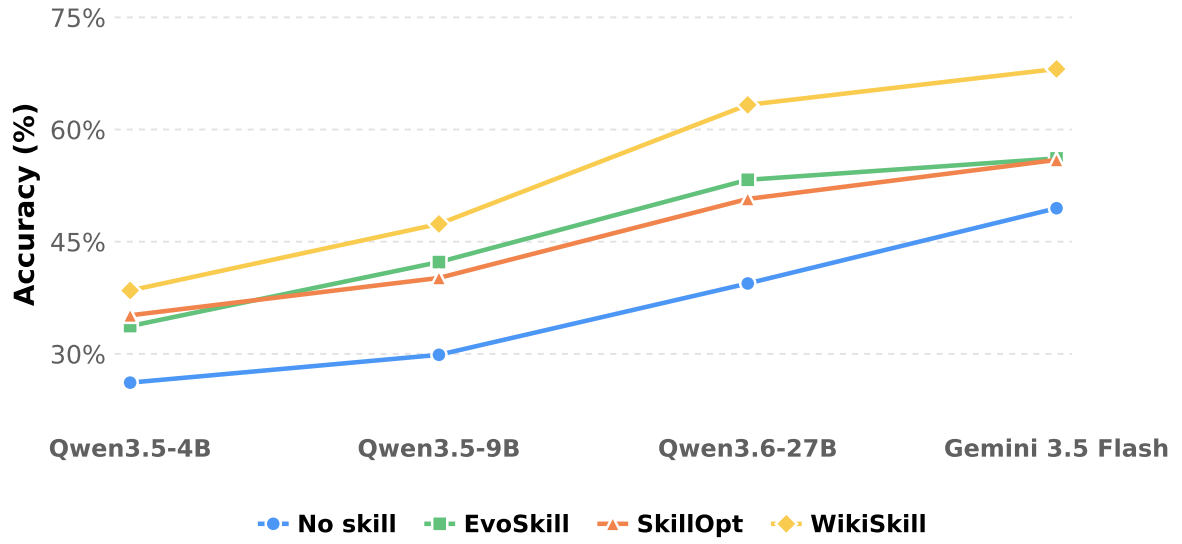


图 1 | **与无技能基线和现有技能演化方法相比，WikiSkill 均能带来稳定提升。**值得注意的是，模型越强，其优势越明显。我们报告了各模型在不使用技能以及使用 EvoSkill、SkillOpt 和 WikiSkill 所演化技能时，在所有评估基准上的平均准确率（详见表 1）。

1. 引言

通用 AI 智能体执行跨领域复杂任务的能力日益增强 (Jackson et al., 2025; Merrill et al., 2026; Patwardhan et al., 2026; Phan et al., 2026)。然而，要可靠地完成现实世界中的任务，往往需要特定领域的专业知识（例如程序性知识和工作流）。智能体技能 (Chen et al., 2026a; Li et al., 2026; Liu et al., 2026; Zhang et al., 2025) 提供了一种轻量、开放的格式，无须更新模型参数即可捕获此类专业知识。技能的核心是将指令、脚本及其他资源封装进一个基于文件系统的可复用模块（即一个组织有序的目录）(Anthropic, 2026; Li et al., 2026; Xia et al., 2026b; Zhang et al., 2025)。这一设计

通讯作者：*lytang@google.com, ttvu@google.com*
© 2026 Google。保留所有权利

让专门知识能够在兼容技能的智能体之间保持一致、可审计且可复用。它还支持*渐进式披露* (Jiang et al., 2026)，使智能体在任一时刻仅加载相关内容，从而节省上下文空间。从更广泛的视角看，技能为独立于模型参数的知识累积提供了一种自然机制。

然而，开发有效的技能仍然颇具挑战。大多数智能体技能由人工编写，这要求开发者预先设想智能体将会需要的程序性知识和工作流 (Li et al., 2026; Liang et al., 2026; Xu and Yan, 2026)。这一挑战催生了近期的一系列工作：让智能体执行训练任务，分析成功与失败的轨迹，再根据所得经验改进技能，从而迭代式地开发智能体技能 (Agrawal et al., 2026; Alzubi et al., 2026; Ni et al., 2026; Ouyang et al., 2026; Yang et al., 2026; Yuksekgonul et al., 2025)。

一个关键的设计问题是：如何保存并组织智能体在整个技能演化过程中学到的内容。既有工作以不同方式处理这一问题。EvoSkill (Alzubi et al., 2026) 累积保存先前提议及其评估结果的历史；Trace2Skill (Ni et al., 2026) 从多条执行轨迹中提取并整合经验教训，再将其转化为技能更新；SkillOpt (Yang et al., 2026) 则利用被拒编辑的反馈和按训练轮次提供的元指导。然而，这些方法并未将所学内容维护为独立且不断演化的知识表示。受 Karpathy (2026) 关于 **LLM Wiki** 的观点启发，该观点倡导将经验编译为持久且持续增益的知识，我们提出如下问题：**能否同样将智能体经验编译为持久知识，以支持长期的技能演化？**我们提出 **WikiSkill**，在原始经验与可执行过程（即技能）之间加入一个结构化知识层。借助这一层，技能开发能够在不同迭代间以证据支持日益充分且整合程度日益提高的知识为基础，而不再依赖散落于技能演化产物中的知识。

WikiSkill 将智能体工作区组织为三层：存储不可变执行轨迹的*原始层*、维护结构化知识的 *Wiki 层*，以及容纳持续演化的程序性知识的*技能层*（图 2）。每次迭代包含四个组件：使用当前技能进行轨迹采样的*推理智能体*、将轨迹整合进 Wiki 的 *Wiki 维护者*、利用 Wiki 和轨迹提出技能更新的*技能提议者*，以及保留能够提升验证性能之更新的*门控与回滚机制*。技能更新可以被回滚，但 Wiki 会持续保留，使未来的更新能够建立在累积知识之上。从高层来看，这些组件构成一个持续循环：经验被整合为持久知识，进而支持技能演化。

我们在*五个*基准上评估 WikiSkill，涵盖数学推理（LiveMathematicianBench (He et al., 2026)）、网络搜索（SealQA (Pham et al., 2026)）、电子表格操作（SpreadSheetBench (Ma et al., 2024)）、长上下文文档问答（OfficeQA (Singhvi et al., 2025)）以及交互式具身任务（ALFWorld (Shridhar et al., 2021)），并使用来自 Qwen (Qwen Team, 2026a,b)、Gemma (Gemma Team, 2026) 和 Gemini (Google DeepMind, 2026) 家族的*五个*模型。我们发现，在大多数设定下，WikiSkill 优于现有技能演化方法，也优于无技能方案。值得注意的是，技能演化与模型规模扩展相互补充。在 Qwen 家族中，WikiSkill 分别将 4B、9B 和 27B 模型的平均性能提升了 12.3%、17.5% 和 23.9%，*且增益随模型规模而提高*。与此同时，演化所得技能能够弥补相当大的模型规模差距：配备 WikiSkill 的 Qwen-3.5-9B 胜过不配备技能的 Qwen-3.6-27B（47.4% 对 39.4%）。我们还发现，演化所得技能能够跨模型家族有效迁移，并可能优于自行演化的技能。例如在 ALFWorld 上，Qwen-3.5-9B 使用由 Qwen-3.6-27B 演化得到的技能时达到 70.2%，而使用自身技能时为 63.4%。这些结果表明，技能发现与技能执行是两种不同的能力。最后，我们的分析显示，持久化 Wiki 对于取得这些增益至关重要，支持了我们的假设：跨迭代累积和提炼知识能够改善技能演化。

总而言之，我们的主要贡献如下：

- 我们提出 WikiSkill，这是一个让智能体技能与持久化知识库协同演化的框架；该知识库持续组织和提炼来自智能体经验的知识。
- 我们通过五个基准和五个模型证明，WikiSkill 始终优于现有技能演化方法；消融实验进一步证实了持久化知识累积的重要性。
- 我们系统研究演化所得技能如何与模型能力相互作用，表明技能演化与模型规模扩展相互补充，且演化所得技能能够在不同模型间有效迁移，有时甚至优于自行演化的技能。

综合来看，我们希望本工作能够推动更多基础研究，探索智能体如何从经验中累积、组织和复用知识。

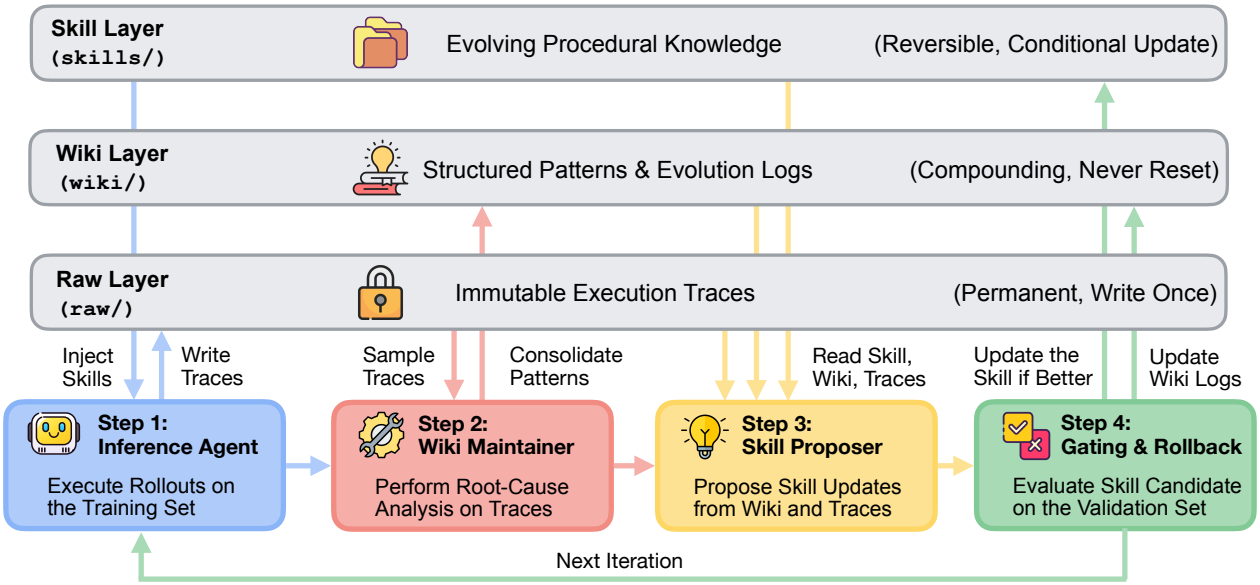


图 2 | **WikiSkill 框架概览**。智能体工作空间分为三层：不可变的执行轨迹（原始层）、跨迭代持续积累的持久化知识库（Wiki 层），以及生效的程序性指令（技能层）。在每次演化循环中，推理智能体在任务上生成执行轨迹（注入生效技能，但限制其访问 Wiki），Wiki 维护者将轨迹整合进 Wiki，技能提议者（采用 ReAct 机制）则提出更新建议；Wiki 会在所有迭代中持续保留。

2. 问题设定

我们对 LLM 智能体的迭代式技能演化任务进行形式化。设 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ 为任务数据集，其中 x_i 表示一个任务实例，y_i 表示其真实答案。我们将 $\mathcal{D}$ 划分为三个互不相交的数据子集：训练任务 $\mathcal{D}_{\text{train}}$、验证任务 $\mathcal{D}_{\text{val}}$ 和测试任务 $\mathcal{D}_{\text{test}}$。

智能体 π 是一个基于 LLM 的系统，配备一组工具 $\mathcal{U}$（例如 Bash 命令行、搜索 API 或文件读取器）以及一个活跃技能集 $S = \{s_1, s_2, \ldots, s_M\}$。技能是一个模块化的、基于文件系统的目录，它将特定领域的程序性知识封装为指令、脚本及其他资源 (Chen et al., 2026a; Li et al., 2026; Liu et al., 2026; Zhang et al., 2025)。具体而言，每项技能均包含一个 SKILL.md 文件，其中既有前置元数据（唯一名称和简洁描述），也有完整的过程指令及适用条件。技能集 S 被初始化为空集（$\emptyset$），并针对每个数据集通过演化过程进行开发。

执行任务 x_i 时，智能体会接收任务上下文 x_i，并获得对可用技能 S 的访问权限。智能体使用工具和技能与环境进行多步交互，从而生成一条执行轨迹 $\tau_i \sim \pi(x_i; S)$。轨迹 $\tau_i = (o_1, a_1, o_2, a_2, \ldots, o_T, a_T)$ 由观测 o_t 和动作 a_t 组成（动作中可能包含对 $\mathcal{U}$ 中工具的调用）。最终动作 a_T 输出预测答案 $\hat{y}_i$。预测的正确性由特定领域的评分函数 $f(\hat{y}_i, y_i) \in [0, 1]$ 评估。对于任意任务子集 $\mathcal{D}_{\text{split}} \subset \mathcal{D}$，让智能体 $\pi(\cdot; S)$ 在 $\mathcal{D}_{\text{split}}$ 的所有任务实例上执行，即可得到对应的执行轨迹集合 $\mathcal{T}_{\text{split}} = \{\tau_i \sim \pi(x_i; S)\}_{(x_i, y_i) \in \mathcal{D}_{\text{split}}}$。任务子集上的性能 $\mathcal{R}(\mathcal{T}_{\text{split}})$ 是所有任务实例 $(x_i, y_i) \in \mathcal{D}_{\text{split}}$ 的平均得分。

在 WikiSkill 中，第 k 次迭代的系统状态由元组 (S_k, W_k) 表示，其中 $S_k = \{s_1, \ldots, s_M\}$ 表示活跃的程序性技能集，W_k 表示持久化知识库（Wiki）。候选技能更新需要经过验证门控，并在得分下降时回滚；而知识库 W_k 则跨迭代持续保留并累积增益。从 $(S_0, W_0) = (\emptyset, \emptyset)$ 出发，WikiSkill 在 $k \in \{1, \ldots, K\}$ 的多次迭代中协同演化联合状态 (S_k, W_k)，利用训练轨迹 $\mathcal{T}_{\text{train},k}$、模式整合以及基于 $\mathcal{T}_{\text{val},k}$ 的验证门控，最大化在未见任务 $\mathcal{D}_{\text{test}}$ 上的最终测试性能 $\mathcal{R}(\mathcal{T}_{\text{test}})$。

3. 方法

我们提出 WikiSkill ，这是一个协同演化智能体技能与持久化知识库（Wiki）的框架。WikiSkill 以三层知识架构（§3.1）为基础，执行一个经过编排的演化循环：推理智能体进行轨迹采样，Wiki 维护者整合轨迹并更新 Wiki，技能提议者提出技能更新，门控机制则筛选这些变更（§3.2）。

3.1. 三层知识架构

WikiSkill 工作区由三个相互区分的层构成，如图 2 所示，具体如下。

原始层（raw/） 该层存储每次迭代中从训练样例收集的原始执行轨迹 $\tau_i \in \mathcal{T}_{\text{train},k}$。这些轨迹记录智能体完整的逐步交互，包括推理、工具调用、工具调用输出以及最终答案。在我们的设定中，Wiki 维护者和技能提议者可以访问这些原始轨迹，以分析智能体行为。为保留原始历史，该层不可修改。

Wiki 层（wiki/） 该层将原始轨迹编译为结构化、可持续增益的知识，并在整个技能演化过程中持续维护。其中包含一个模式目录（patterns/），目录内的独立 Markdown 文件记录具体的失败模式或成功策略，以及可执行的规避方案。关键在于，该层通过演化日志（logs.md，由 Wiki 维护者更新）和技能影响跟踪器（skill-impact.md，在验证门控后由外循环编排框架以程序方式更新），提供跨优化迭代的长期历史认知。这些记录使 Wiki 维护者和技能提议者能够：（1）查看完整的技能接受历史，从而不再重复提出已被拒绝的干预；（2）跟踪先前迭代提出过哪些方案，以及这些方案是否成功；（3）识别哪些错误在不同迭代中反复出现。Wiki 不会在迭代之间重置，而是在整个演化过程中持续累积并编译知识。

技能层（skills/） 该层包含演化所得技能的活跃集合 S，这些技能对推理智能体可读取的程序性知识进行编码。WikiSkill 中的每个技能目录都包含两个文件：SKILL.md，保存技能的完整内容；以及 PURPOSE.md，将技能追溯到促成其创建或修改的 Wiki 模式。图 3 展示了技能层与 Wiki 层交互的详细示例，第 5.3 节的案例研究对其作了说明。

3.2. 演化智能体与 Wiki 编排

WikiSkill 循环由四个组件构成。在每次迭代中，**推理智能体**（§3.2.1）使用 skills/ 中的活跃技能执行任务，并在 raw/ 中生成不可修改的执行轨迹。在训练轨迹采样期间，推理智能体不得访问 Wiki 层，因为我们的消融研究（§5.1）表明，在训练时允许访问 Wiki 会对技能开发产生负面影响。接下来，**Wiki 维护者**（§3.2.2）结合现有 wiki/ 层分析这些原始轨迹，以诊断失败并提取成功策略，同时更新持久模式目录和演化日志。随后，**技能提议者**（§3.2.3）审阅更新后的 Wiki，并读取最新一次迭代的执行轨迹，以在 skills/ 中生成或修改候选技能。最后，**门控与回滚机制**（§3.2.4）在验证集上评估候选技能，接受成功的修改；若变更导致性能下降，则回滚技能集。完整的演化算法见附录 A 中的算法 1。

3.2.1. 为推理智能体配置技能

在第 k 次迭代中，推理智能体 π 以活跃技能集 S_{k-1} 为条件，并使用环境工具 $\mathcal{U}$ 执行一条多轮轨迹：

$$\tau_i \sim \pi(x_i; S_{k-1}) \tag{1}$$

在 WikiSkill 中，活跃技能 S_{k-1} 的完整内容会被直接注入推理智能体的系统提示词。沿用既有工作 (Ni et al., 2026; Yang et al., 2026)，这种完整注入设定确保程序性指令在任务执行期间可以立即使用，从而排除技能触发失败或检索失败对本研究造成的混淆。

3.2.2. *Wiki* 维护者：模式整合

在第 k 次迭代中，我们先在训练集 $\mathcal{D}_{\text{train}}$ 上获得轨迹采样结果 $\mathcal{T}_{\text{train},k}$，随后从成功和失败的执行轨迹中抽取一个子集 $\mathcal{T}_{\text{sample},k} \subset \mathcal{T}_{\text{train},k}$（采样预算和分层标准见附录 C），以避免受到上下文窗口限制。Wiki 维护者 $\mathcal{M}_{\text{WM}}$ 将这些观测整合进持久化 Wiki W_{k-1}，产生中间 Wiki 状态 W'_k：

$$W'_k \leftarrow \mathcal{M}_{\text{WM}}(W_{k-1}, \mathcal{T}_{\text{sample},k}) \tag{2}$$

Wiki 维护者会同时接收完整的 Wiki 上下文 W_{k-1} 与采样轨迹 $\mathcal{T}_{\text{sample},k}$。它对失败任务进行根因分析，并从通过的任务中提取成功策略。在每次迭代中，Wiki 维护者可以在 wiki/patterns/ 下创建新的模式页面，并用新证据或经过提炼的解决方案更新既有模式页面。模式页面的更新采用基于补丁的增量编辑方式（例如追加、替换或插入文本片段）。每当模式发生修改，Wiki 维护者都会修订 index.md 目录以反映当前状态，并将本次迭代发现的摘要追加到演化日志 logs.md。每次迭代中创建或更新的模式数量没有硬性上限；Wiki 维护者会依据轨迹和当前 Wiki 状态，决定哪些更新是必要的。

3.2.3. 基于 *Wiki* 的技能提议者

技能提议者 $\mathcal{M}_{\text{P}}$ 是一个基于 LLM 的智能体，负责技能发现与提炼。在第 k 次迭代中，技能提议者以多轮 ReAct 风格 (Yao et al., 2023) 工作。为了在分析较长执行历史时避免耗尽上下文窗口，我们不会向技能提议者提供一组固定的预采样轨迹；相反，初始时会向其提供 Wiki 索引 $I(W'_k)$、历史技能影响跟踪器（skill-impact.md）以及所有训练任务结果的简明摘要（通过/失败状态、预测答案和真实答案）。作为一个自主智能体，它会主动推理并使用环境工具（read_file），按需选择和检查特定模式页面及原始执行轨迹 $\tau_i \in \mathcal{T}_{\text{train},k}$，在综合形成提议 P_k 之前诊断根因：

$$P_k \leftarrow \mathcal{M}_{\text{P}}(W'_k, S_{k-1}, \mathcal{T}_{\text{train},k}) \tag{3}$$

在每次迭代中，技能提议者都会生成一个只针对单项技能的原子提议 P_k，即创建一项新技能，或对目标既有技能执行基于补丁的增量编辑。

3.2.4. 门控与回滚

提议 P_k 生成后，它会被应用到工作区，得到候选技能集 $S'_k = \text{Apply}(S_{k-1}, P_k)$。系统在验证集 $\mathcal{D}_{\text{val}}$ 上评估 S'_k，得到验证轨迹 $\mathcal{T}_{\text{val},k}$ 和得分 $\mathcal{R}(\mathcal{T}_{\text{val},k})$。接受决策遵循：

$$S_k \leftarrow \begin{cases} S'_k & \text{if } \mathcal{R}(\mathcal{T}_{\text{val},k}) > \mathcal{R}_{\text{best}} \\ S_{k-1} & \text{otherwise} \end{cases} \tag{4}$$

若候选技能被接受，则将其保留为新的活跃技能集 S_k，并将基准性能阈值 $\mathcal{R}_{\text{best}}$ 更新为 $\mathcal{R}(\mathcal{T}_{\text{val},k})$。在演化循环开始前，$\mathcal{R}_{\text{best}}$ 被初始化为基线验证得分 $\mathcal{R}(\mathcal{T}_{\text{val},0})$；该得分通过在 $\mathcal{D}_{\text{val}}$ 上评估空技能集 S_0 获得。若验证得分在演化过程中的任一时刻达到最大值（$\mathcal{R}_{\text{best}} = 1.0$），演化循环便会提前终止。若候选技能被拒绝，系统会丢弃候选技能修改，并将技能集恢复为最近一次成功的配置 S_{k-1}。值得注意的是，无论接受决策如何，Wiki W_k 都不会回滚；累积的模式和日志会在所有迭代间持续保留，以确保长期知识留存。每次验证评估后，外循环编排框架都会通过 $W_k \leftarrow \text{Update}(W'_k, P_k, \mathcal{R}(\mathcal{T}_{\text{val},k}), a_k)$，以程序方式向 wiki/skill-impact.md 追加一条记录，其中包含提议元数据、目标技能名称、修改的统一差异、验证得分 $\mathcal{R}(\mathcal{T}_{\text{val},k})$，以及最终接受结果 $a_k \in \{\text{Accepted}, \text{Rejected}\}$。至此，第 k 次迭代的 Wiki 状态转换 $W_{k-1} \to W_k$ 完成，同时形成一条关于过往干预的客观、真实可核验的审计轨迹，供技能提议者在后续迭代中查阅，以避免重复失败的修改。

4. 实验与结果

4.1. 实验设置

数据集 我们在涵盖不同领域的五个基准上进行评估：数学推理（**LiveMathematicianBench (LiveMath)** (He et al., 2026)）、网络搜索（**SealQA** (Pham et al., 2026)）、电子表格操作（**SpreadsheetBench (SpreadSheet)** (Ma et al., 2024)）、长上下文文档问答（**OfficeQA** (Singhvi et al., 2025)）以及交互式具身任务（**ALFWorld** (Shridhar et al., 2021)）。数据集详情与统计信息见附录 B。

基线 我们将 WikiSkill 与三种有代表性的技能演化基线进行比较，包括 Trace2Skill (Ni et al., 2026)、EvoSkill (Alzubi et al., 2026) 和 SkillOpt (Yang et al., 2026)。这些方法都采用相同的一般循环：运行智能体、分析执行轨迹、提出技能修改，并通过验证决定是否接受修改。我们还评估了各模型不使用技能的情况，作为无技能基线。附录 D 给出了这些框架的详细说明以及优化器 API 调用复杂度分析。我们的比较聚焦于专用技能演化框架，而非通用自动提示词优化器（例如 GEPA (Agrawal et al., 2026)），因为既有工作表明，专用技能演化流水线始终优于通用提示词优化方法 (Yang et al., 2026)。

模型 我们同时使用闭源模型和开放权重模型来评估 WikiSkill 及各基线。闭源模型采用 **Gemini-3.5-Flash** (Google DeepMind, 2026)。开放权重模型评估 **Qwen-3.5-4B/9B-Instruct** (Qwen Team, 2026a)、**Qwen-3.6-27B** (Qwen Team, 2026b) 和 **Gemma-4-31B-It** (Gemma Team, 2026)，并使用 vLLM 框架 (Kwon et al., 2023) 部署这些模型。

4.2. 主要结果

我们在不同模型和任务上评估 WikiSkill ，并研究演化出的技能能否跨模型迁移。表 1 给出了不同模型与任务上的主要技能演化结果，包括技能演化收益随模型规模变化的情况；表 2 则给出了跨模型技能迁移结果。下文将详细分析这些结果。

为考虑结果波动，我们对每种方法独立运行三次完整演化过程；所有报告分数均表示由此得到的三个演化技能集的平均测试性能。性能差异的统计显著性使用配对自助法检验评估，阈值为 $p < 0.05$（附录 C）。需要注意的是，在 ALFWorld 上，Gemini-3.5-Flash 的所有演化方法都与无技能基线取得相同性能（85.9%），因为 Gemini-3.5-Flash 在技能演化前就在验证集（$\mathcal{D}_{\text{val}}$）上获得了 100% 的分数。这也解释了为何在跨模型迁移评估（表 2）中，Gemini-3.5-Flash 作为 ALFWorld 的技能来源时被标记为“–”。

4.2.1. 跨模型与任务的技能演化

WikiSkill 在不同模型和数据集上均带来稳定提升 如表 1 所示，WikiSkill 在全部五个模型上都取得了最高的平均性能。与每个模型上最强的竞争性技能演化方法相比，WikiSkill 在 Qwen-3.5-4B、Qwen-3.5-9B、Qwen-3.6-27B、Gemma-4-31B 和 Gemini-3.5-Flash 上分别将平均性能提升了 3.3、5.1、10.0、5.8 和 12.0 个百分点。这些提升在不同设置下具有一致性：对于大多数模型–数据集组合，WikiSkill 均优于无技能基线；在五个数据集的平均性能上，也对所有模型都达到或超过了最强的竞争方法。性能提升还覆盖多个不同领域，且幅度可观。例如，在 LiveMath 上，WikiSkill 将 Gemini-3.5-Flash 的性能从 33.0% 提升至 72.6%；在 SpreadSheet 上从 50.5% 提升至 76.6%；同时在 ALFWorld 上将 Qwen-3.6-27B 的性能从 52.8% 提升至 77.6%。相比之下，现有技能演化方法的一致性较差。例如，EvoSkill 在 LiveMath 上显著提升了 Qwen-3.5-9B（28.2% → 58.1%），却降低了 Gemma-4-31B 在同一基准上的性能（33.9 % → 29.8%）；SkillOpt 则降低了 Gemini-3.5-Flash 在 SealQA 上的性能（29.4 % → 28.2%）。这些结果表明，WikiSkill 在不同设置下都能带来更强且更可靠的提升。

Model	Method	LiveMath	SealQA	SpreadSheet	OfficeQA	ALFWorld	Avg.
Qwen-3.5-4B	No skill	29.1	32.5	14.6	30.2	24.4	26.2
	Trace2Skill	31.5	**37.6**	**17.5**	31.0	42.8	32.1
	EvoSkill	41.7	**37.3**	**18.6**	29.5	41.5	33.7
	SkillOpt	**48.7**	33.3	14.0	34.5	45.3	**35.2**
	WikiSkill	**49.7**	**39.4**	**21.1**	28.5	**53.7**	**38.5**
Qwen-3.5-9B	No skill	28.2	26.3	24.3	35.9	34.7	29.9
	Trace2Skill	33.1	36.9	26.5	38.4	48.8	36.7
	EvoSkill	**58.1**	34.5	**35.4**	34.9	48.5	42.3
	SkillOpt	48.7	29.4	29.0	38.0	55.7	40.2
	WikiSkill	**56.3**	**43.1**	**33.6**	40.5	**63.4**	**47.4**
Qwen-3.6-27B	No skill	33.9	27.5	40.8	42.1	52.8	39.4
	Trace2Skill	36.3	37.3	53.3	**54.3**	55.5	47.3
	EvoSkill	**57.3**	32.9	59.5	**52.5**	64.2	53.3
	SkillOpt	51.9	34.5	53.2	**54.8**	59.2	50.7
	WikiSkill	**61.9**	**41.6**	**81.7**	**53.7**	**77.6**	**63.3**
Gemma-4-31B	No skill	33.9	30.6	48.3	43.3	50.4	41.3
	Trace2Skill	32.3	37.7	58.5	43.2	57.2	45.8
	EvoSkill	29.8	**38.4**	56.4	39.9	52.6	43.4
	SkillOpt	40.1	36.1	**63.1**	44.4	61.9	49.1
	WikiSkill	**56.7**	**41.2**	**68.0**	44.2	**64.4**	**54.9**
Gemini-3.5-Flash	No skill	33.0	29.4	50.5	48.6	85.9	49.5
	Trace2Skill	41.9	**44.3**	56.0	50.0	85.9	55.6
	EvoSkill	44.6	**43.6**	55.4	51.2	85.9	56.1
	SkillOpt	49.7	28.2	66.1	49.8	85.9	55.9
	WikiSkill	**72.6**	**44.7**	**76.6**	**60.7**	85.9	**68.1**

表 1 | **不同推理模型和测试集上的方法比较。**每个横向区块评估一个特定推理模型在不使用技能（No skill）以及使用不同技能演化方法所开发技能时的表现。为确保公平比较，所有技能演化方法均从空技能集开始，并在推理时将演化出的技能注入推理智能体的提示词中。所有报告分数均为完整演化过程三次独立运行所得测试性能的平均值。高亮行表示我们的方法（**WikiSkill**）。粗体表示各数据集上的最佳性能；若有多个粗体结果，则表示以 $p < 0.05$ 为显著性阈值、进行 1,000 次迭代的配对自助法检验后，这些方法与最佳方法之间不存在显著差异。

技能演化的收益随模型能力增强而增长，并与模型规模扩展形成互补 在 Qwen 系列内部，WikiSkill 带来的平均提升随模型规模增大而增加：Qwen-3.5-4B 为 +12.3 个百分点，Qwen-3.5-9B 为 +17.5 个百分点，Qwen-3.6-27B 则为 +23.9 个百分点。这一趋势在 SpreadSheet 上尤为明显，WikiSkill 对三个模型的提升分别为 +6.5、+9.3 和 +40.9 个百分点，说明技能演化的收益可能随模型规模增大而显著增加。与此同时，演化出的技能可以弥补模型规模上的巨大差距：采用 WikiSkill 的 Qwen-3.5-9B 达到 47.4% 的平均准确率，超过未使用技能、平均准确率为 39.4% 的 Qwen-3.6-27B；采用 WikiSkill 的 Qwen-3.5-4B 也达到 38.5%。我们的结果表明，模型能力与演化出的程序性知识是两种相互补充的性能来源：更强的模型可以通过开发并执行更有效的技能，从技能演化中获得更大价值；而有效的技能可以让较小模型胜过不使用技能、规模大得多的模型。

技能演化的收益在不同数据集之间也有显著差异 我们的结果表明，一些数据集比另一些数据集更适合技能演化。对于 Qwen-3.6-27B，WikiSkill 在 OfficeQA 和 SealQA 上分别提升 11.6 和 14.1 个百分点，而在 ALFWorld、LiveMath 和 SpreadSheet 上则分别提升 24.8、28.0 和 40.9 个百分点。其他模型上也出现了类似差异。LiveMath 始终能从技能演化中获益，五个模型上的增益介于 20.6 至 39.6 个百分点；对于 WikiSkill 实际演化技能的四个模型，ALFWorld 上的增益介于 14.0 至 29.3 个百分点（Gemini-3.5-Flash 因提前停止而不计入）。相比之下，OfficeQA 的长上下文文档检索要求带

Model	Skill Source	LiveMath	SealQA	SpreadSheet	OfficeQA	ALFWorld
Qwen-3.5-4B	None	29.1	32.5	14.6	30.2	24.4
	Qwen-3.5-4B	49.7	**39.4**	21.1	28.5	**53.7**
	Qwen-3.6-27B	59.7	**38.8**	**33.0**	25.4	**57.0**
	Gemini-3.5-Flash	**62.6**	**37.3**	23.0	32.2	-
Qwen-3.5-9B	None	28.2	26.3	24.3	35.9	34.7
	Qwen-3.5-4B	**61.0**	**40.4**	25.0	40.3	**69.2**
	Qwen-3.5-9B	56.3	**43.1**	33.6	40.5	63.4
	Qwen-3.6-27B	59.1	**40.4**	**50.5**	39.9	**70.2**
	Gemini-3.5-Flash	53.0	**39.6**	**48.8**	40.5	-
Qwen-3.6-27B	None	33.9	27.5	40.8	42.1	52.8
	Qwen-3.5-4B	**62.6**	41.6	40.6	**52.9**	72.1
	Qwen-3.6-27B	**61.9**	41.6	**81.7**	**53.7**	**77.6**
	Gemini-3.5-Flash	**65.1**	**51.0**	76.0	**52.5**	-
Gemma-4-31B	None	33.9	30.6	48.3	43.3	50.4
	Qwen-3.5-4B	**73.1**	**38.8**	37.1	42.1	**66.9**
	Qwen-3.6-27B	**73.7**	37.7	**72.0**	44.2	**66.9**
	Gemma-4-31B	56.7	**41.2**	**68.0**	44.2	64.4
	Gemini-3.5-Flash	61.8	37.7	**68.8**	43.4	-
Gemini-3.5-Flash	None	33.0	29.4	50.5	48.6	85.9
	Qwen-3.5-4B	67.5	40.0	18.1	48.5	87.3
	Qwen-3.6-27B	**73.9**	**43.5**	63.4	47.7	86.8
	Gemini-3.5-Flash	**72.6**	**44.7**	**76.6**	**60.7**	-

表 2 | **跨模型技能迁移结果。**我们评估推理模型不使用技能（None）以及使用由 WikiSkill 分别以 Qwen-3.5-4B、Qwen-3.6-27B 和 Gemini-3.5-Flash 为来源模型演化出的技能时的表现。推理时，技能会被注入推理智能体的系统提示词。高亮行表示自演化技能，即推理模型与技能来源相同。每个模型区块内，各基准上的最高性能以粗体表示。“–”表示来源模型在技能演化前已达到 100% 的验证性能，因此没有演化出技能。

来了独特挑战。较大模型能够有效利用演化出的搜索工作流在长篇文档中导航（例如 Qwen-3.6-27B 提升 +11.6 个百分点，Gemini-3.5-Flash 提升 +12.1 个百分点）；而 Qwen-3.5-4B 难以在长上下文中执行这些多步搜索工作流，会退回到其默认阅读行为，从而导致性能略有下降。

4.2.2. 使用 *WikiSkill* 的跨模型技能迁移

演化出的技能能够有效跨模型迁移，且迁移技能可以优于自演化技能 表 2 评估了使用不同来源模型开发时，WikiSkill 演化出的技能如何在推理模型之间迁移。迁移技能经常同时优于无技能基线和自演化技能。例如，Qwen-3.6-27B 的技能使 Qwen-3.5-9B 在 SpreadSheet 上提升至 50.5%，而无技能和使用自演化技能时的性能分别为 24.3% 和 33.6%；该技能还使 Gemma-4-31B 在 LiveMath 上提升至 73.7%，而另两种设置的性能分别为 33.9% 和 56.7%。值得注意的是，从较小模型到较大模型也能实现有效迁移：Qwen-3.5-4B 的技能使 Gemma-4-31B 在 LiveMath 和 ALFWorld 上分别达到 73.1% 和 66.9%。我们的结果表明，更强的来源模型未必会生成更好的技能，并且一个模型从自身经验中开发出的程序性知识可以跨模型规模和模型家族迁移。

演化技能的可迁移性取决于其捕获的是通用流程还是模型特定的变通策略 表 2 中的结果表明，WikiSkill 既能产生可跨模型迁移的通用程序性知识，也可能产生会导致负迁移的模型特定策略。LiveMath 技能的跨模型迁移效果尤其好：Qwen-3.5-4B 和 Qwen-3.6-27B 的技能分别将 Gemini-3.5-Flash 的性能从 33.0% 提升至 67.5% 和 73.9%。相比之下，SpreadSheet 表现出很强的来源–目标交互效应。Qwen-3.5-4B 的技能将 Gemini-3.5-Flash 的性能从 50.5% 降至 18.1%，而 Qwen-3.6-27B 的

WikiSkill Components		Benchmarks				
Inference Agent Wiki Access	Skill Proposer Wiki Access	LiveMath	SealQA	SpreadSheet	OfficeQA	Avg.
No skill		33.0	29.4	50.5	48.6	40.4
✓	✗	43.8	42.0	44.4	51.0	45.3
✗	✗	51.3	38.4	49.9	55.2	48.7
✓	✓	64.8	42.8	**80.2**	55.6	60.9
✗	✓	**72.6**	**44.7**	76.6	**60.7**	**63.7**

表 3 | **使用 Gemini-3.5-Flash 对 WikiSkill 进行的消融研究。**我们在四种配置下评估各基准上的性能，这些配置改变了技能演化期间推理智能体和技能提议者是否能够访问 Wiki。当技能提议者无法访问 Wiki 时，我们也会移除 Wiki 维护者，以消除跨迭代的持久化知识积累。最底行表示 WikiSkill 的默认配置。

技能则将其提升至 63.4%。我们的错误分析发现了造成这种负迁移的两个因素。首先，Qwen-3.5-4B 的技能编码了底层变通策略，例如单行 Python 命令和字符串转换规则；这些策略有助于较小模型避免执行失败，却会限制 Gemini-3.5-Flash 等较强模型使用完整的端到端脚本。其次，碎片化的诊断流程会引入冗余工具调用，可能在任务完成前耗尽 Gemini-3.5-Flash 的交互预算。

迁移技能的效用还取决于推理模型执行这些技能的能力 下面我们考察不同推理模型如何使用由同一来源模型开发的技能。在 Qwen 家族内部，更强的模型可以从同一份程序性知识中获得更大价值。例如，相比各自的无技能基线，Qwen-3.6-27B 的 SpreadSheet 技能使 Qwen-3.5-4B、Qwen-3.5-9B 和 Qwen-3.6-27B 分别提升 18.4%、26.2% 和 40.9%。OfficeQA 展示了这样一个案例：某个模型开发的技能对另一个模型比对其自身更有用。Qwen-3.5-4B 的技能使其自身性能从 30.2% 降至 28.5%，却将 Qwen-3.6-27B 从 42.1% 提升至 52.9%。我们的轨迹分析表明，在长上下文设置中，较小模型可能受到冗长文档上下文的干扰，无法遵循详细的多步搜索指令，转而退回其默认的文档阅读行为。相比之下，较强模型能在长上下文中更可靠地执行技能所规定的结构化导航流程。综合来看，这些结果区分了自演化通常混为一谈的两种能力：从经验中发现有用程序性知识，以及在推理时有效执行这些知识。

5. 分析与讨论

5.1. 持久化知识在技能演化中的作用

为理解持久化知识在哪些环节促进了技能演化，我们对演化期间能够使用 Wiki 的两个组件进行 Wiki 访问消融（表 3）。具体而言，我们使用 Gemini-3.5-Flash，分别改变训练运行中推理智能体和技能开发中技能提议者的 Wiki 访问权限，从而得到四种配置。当技能提议者无法访问 Wiki 时，我们也会移除 Wiki 维护者，以消除跨迭代的持久化知识积累。WikiSkill 的默认配置允许技能提议者访问 Wiki，但不允许推理智能体访问。

持久化 Wiki 知识显著改善技能演化 如表 3 所示，当推理智能体的 Wiki 访问权限被禁用时，允许技能提议者访问持久化 Wiki，可将各基准的平均性能从 48.7% 提升至 63.7%（+15.0 个百分点），其中 LiveMath（51.3% 提升至 72.6%）和 SpreadsheetBench（49.9% 提升至 76.6%）上的增益尤为显著。如果缺少跨迭代积累的持久化知识，技能提议者就难以解决复杂的失败模式。

演化期间允许推理智能体访问 Wiki 会降低最终技能质量 当技能提议者能够访问持久化 Wiki 时，在训练运行期间允许推理智能体访问 Wiki，会使各基准的平均性能从 63.7% 降至 60.9%，其中 LiveMath 上的性能从 72.6% 显著降至 64.8%。我们推测，当推理智能体在训练运行期间既能访问

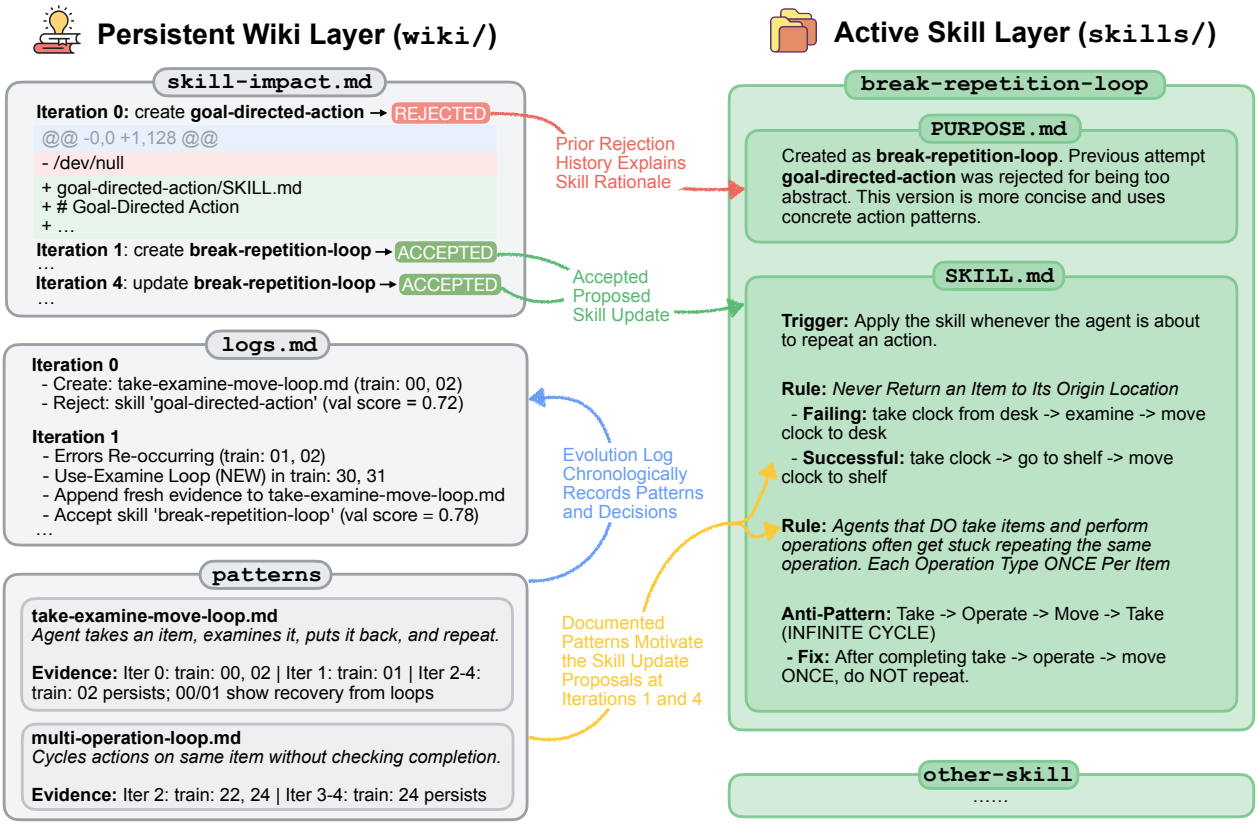


图 3 | ALFWorld（Qwen-3.6-27B）上 Wiki 引导的技能演化案例研究。持久化 Wiki 层汇集了跨迭代模式、过往提议差异与接受决策的审计轨迹，以及按时间顺序记录的历史。基于迭代 0 中技能提议被拒绝的结果，技能提议者在迭代 1 中综合形成了一项被接受的技能更新，并在此后依据新的模式证据进一步精炼该技能。为便于理解，图中的文件内容经过简化。

技能又能访问 Wiki 时，一部分任务求解知识可能直接来自 Wiki 而非技能，因而生成的轨迹对技能开发所提供的信息量会减少。

5.2. 定性分析：技能与 Wiki 的动态变化

为更好地理解 WikiSkill 如何在不同模型和数据集上演化知识与技能，我们分析了演化期间积累的 Wiki 模式和生成的技能（表 4），以及成功的技能更新在各次迭代中被接受的时间分布（附录表 5）。

WikiSkill 在生成简洁技能的同时持续积累 Wiki 模式 表 4 汇总了不同模型和基准上的技能与 Wiki 模式创建、编辑情况及其平均长度。从模型维度看，Qwen 模型生成的程序性技能较长（118.9–128.6 行），而 Gemma-4-31B 和 Gemini-3.5-Flash 生成的技能更紧凑（分别为 45.1 行和 81.2 行）。Wiki 模式的积累也因模型而异，平均创建 6.3–8.9 个模式，并进行 7.0–18.4 次编辑。从基准维度看，SpreadSheet 生成的技能最长（142.5 行），Wiki 模式也最多（9.8 个）；而 LiveMath 生成的技能最短（84.6 行），Wiki 模式也最少（4.4 个）。总体而言，这些结果表明，技能结构和 Wiki 积累都会随模型和数据集而变化。

技能精炼贯穿整个演化过程 附录表 5 将被接受的技能更新分为早期（迭代 0–1）、中期（迭代 2–4）和后期（迭代 5–7）三个阶段。从模型维度看，初始阶段占被接受更新的 39%–52%，另有相当比例的更新持续到中期和后期。从基准维度看也存在类似模式，39%–58% 的被接受更新发生在初始阶

Category	Skills			Wiki Patterns		
	Create (Proposed / Accepted)	Edit (Proposed / Accepted)	Avg. Length	Create	Edit	Avg. Length
By Model (All-Dataset Average)						
Qwen-3.5-4B	3.1 / 1.6	4.9 / 1.3	126.2	8.8	18.4	48.2
Qwen-3.5-9B	4.6 / 1.4	3.4 / 0.7	128.6	7.3	10.9	26.6
Qwen-3.6-27B	4.4 / 1.5	3.6 / 0.8	118.9	6.5	17.9	47.7
Gemma-4-31B	4.8 / 1.3	3.2 / 0.8	45.1	6.3	13.7	23.7
Gemini-3.5-Flash	2.3 / 1.2	5.7 / 1.1	81.2	8.9	7.0	18.1
By Benchmark (All-Model Average)						
LiveMath	1.9 / 1.1	6.1 / 1.9	84.6	4.4	12.1	31.7
SealQA	4.9 / 0.9	3.1 / 0.4	98.5	9.4	15.9	26.9
SpreadSheet	4.5 / 1.4	3.5 / 1.1	142.5	9.8	11.3	38.5
OfficeQA	4.7 / 1.8	3.3 / 0.3	102.9	8.3	14.9	31.4
ALFWorld	3.9 / 1.6	4.1 / 0.8	93.5	5.8	14.3	40.6

表 4 | **不同推理模型（上半部分）和基准（下半部分）上的演化技能与 Wiki 模式统计。**对于技能，我们报告提出/接受的创建与编辑次数，以及以 Markdown 行数计的平均长度。对于 Wiki 模式，我们报告创建次数、编辑次数及以 Markdown 行数计的平均长度。所有 Wiki 模式的创建与编辑均予保留。

段。SealQA 上的持续精炼尤为明显：33% 的被接受更新发生在中期，28% 发生在后期。结合第 5.1 节中的消融结果，这些结果表明，持久化知识积累支持技能在多次迭代中持续精炼。Wiki 层会保存反复出现的错误、被拒绝的提议和演化历史，为技能提议者的后续更新提供累积上下文。下面，我们通过一个案例研究说明这些累积知识如何指导技能演化。

5.3. 案例研究：Wiki 引导的技能演化过程剖析

为说明 Wiki 层与技能层在演化期间如何交互，我们追踪了 Qwen-3.6-27B 在 ALFWorld 上的一个具体示例，如图 3 所示（为便于展示进行了简化）。

在迭代 0 中，Wiki 维护者识别出一种基本循环行为（take-examine-move-loop.md），技能提议者则提出了 goal-directed-action；由于该提议未能提升验证集性能，因而被拒绝。关键在于，*skill-impact.md* 保存了提议差异及拒绝结果，使后续技能更新能够考虑这次失败尝试。

在这条审计轨迹的指导下，技能提议者在迭代 1 中创建了 break-repetition-loop，其中包含一条具体行动规则（*切勿将物品放回其原始位置*），并获接受。随着多次运行中出现新的循环变体（multi-operation-loop.md），Wiki 维护者会在持久化 Wiki 中积累新证据。在这些累积的 Wiki 模式和新存储的轨迹（图中未显示）指导下，技能提议者在迭代 4 中通过一条新规则（*每件物品的每种操作类型只执行一次*）进一步精炼该技能。这个示例说明了先前迭代中的持久化知识如何指导后续技能精炼。

6. 相关工作

经验驱动的智能体技能演化 智能体技能编码了可复用的程序性知识，使 LLM 智能体能够将过往经验用于未来任务 (Anthropic, 2026; Li et al., 2026; Wang et al., 2026; Xia et al., 2026b; Xu and Yan, 2026; Zhang et al., 2025; Zhou et al., 2026)。近期框架使智能体能够通过从以往执行轨迹中发现并精炼程序性知识来实现自我改进 (Agrawal et al., 2026; Lu et al., 2026; Ouyang et al., 2026; Xia et al., 2026a; Yuksekgonul et al., 2025)。EvoSkill (Alzubi et al., 2026)、Trace2Skill (Ni et al., 2026) 和 SkillOpt (Yang et al., 2026) 等方法使用专用智能体流水线分析任务运行并更新模块化技能文档 (Zhang et al., 2026a)。然而，这些方法并未将已学知识维护为一种独立且不断演化的知识

表示。WikiSkill 引入持久化 Wiki 层，在多次迭代中将经验整合为结构化知识，使后续技能更新能够以累积知识为基础进行系统构建。

技能增强型智能体与智能体自我改进 除构建高质量技能外，技能增强型智能体还必须在执行期间有效选择并利用相关技能 (Chen et al., 2026a; Liu et al., 2026)。随着可复用技能数量增加，近期工作开始探索技能检索，以便从技能库中为每项任务选择相关技能 (Cho et al., 2026; Shi et al., 2026; Su et al., 2026; Ye et al., 2026; Zheng et al., 2026)。与之不同，WikiSkill 将关注点放在技能质量本身，并将其与技能检索分开研究。另一条研究路线优化更广泛的智能体运行框架，包括提示词、上下文、工具、记忆和工作流 (Chen et al., 2026b; Lee et al., 2026; Lin et al., 2026; Lou et al., 2026; Zhang et al., 2026b)。这些方法通过分析执行轨迹与环境反馈来搜索更优的智能体配置，从而改进智能体系统。这一方向与 WikiSkill 互为补充；后者在保持更广泛的智能体运行框架不变的情况下，专注于演化可复用的程序性技能。

7. 结论

本文提出了 WikiSkill，这是一个让智能体技能与持久化、可累积的知识库（Wiki）共同演化的框架。通过将智能体工作空间划分为三个不同层次，WikiSkill 使技能开发能够在多次迭代中以证据日益充分、整合程度日益提高的知识为基础。一个经过编排的循环会将经验整合进 Wiki，依据累积知识提出技能精炼方案，并根据验证性能决定是否接受修改。实验结果表明，在五个基准和五个推理模型上，WikiSkill 始终优于现有技能演化方法，并在大多数模型–数据集组合上优于无技能基线。除这些总体增益外，技能演化还与模型规模扩展形成互补：较大模型通常能从演化技能中获益更多，而使用技能的较小模型可以胜过规模大得多但不使用技能的模型。与此同时，演化技能可以在不同模型及模型家族之间有效迁移，且能够优于自演化技能。最后，我们的消融实验验证了持久化知识积累对于实现有效技能演化至关重要。

局限性

WikiSkill 存在若干局限，也由此引出了未来工作的方向。第一，为了单独考察技能质量并避免技能检索带来的混杂影响，我们遵循既有工作，将生效技能直接注入智能体提示词。这一设置未评估技能检索或触发；随着可用技能数量增长，这些能力将变得愈发重要。第二，我们的验证门控要求每个被接受的提议都必须提高验证分数，因而排除了那些虽能维持即时性能、却可能为后续迭代带来增益的中性提议。为确保公平比较，我们遵循既有技能演化框架 (Alzubi et al., 2026; Yang et al., 2026)，采用了这一严格标准。探索更灵活的接受标准是未来工作的重要方向。第三，Wiki 层在多次迭代中持续积累模式页面、演化日志和提议差异，但 WikiSkill 目前缺少自动化的 Wiki 剪枝机制。随着知识在更长的演化运行中不断累积，这类剪枝可能会变得必要。最后，尽管我们的基准套件包含长上下文文档推理（OfficeQA）和多步工具交互，却未覆盖跨越数百次环境操作或持续数小时的超长时域任务。开发在线技能自适应方法，使程序性知识能够在一次长时执行过程中得到精炼，仍是未来工作的一个重要方向。

AI 使用披露

本文使用大语言模型和编程智能体来辅助并润色写作，以及生成部分表格和图形。

参考文献

L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA：反思式提示词演化可以胜过强化学习. In 第十四届国际学习表征会议, 2026. URL https://openreview.net/forum?id=RQm2KQTM5r.

S. Alzubi, N. Provenzano, J. Bingham, W. Chen, and T. Vu. Evoskill：面向多智能体系统的自动化技能发现. *arXiv preprint arXiv:2603.02766*, 2026. URL https://arxiv.org/abs/2603.02766.

Anthropic. 为 claude 构建技能的完整指南. https://claude.com/blog/complete-guide-to-building-skills-for-claude, January 2026.

S. Chen, J. Gai, R. Zhou, J. Zhang, T. Zhu, J. Li, K. Wang, Z. Wang, Z. Chen, K. Kaleb, et al. Skillcraft：llm 智能体能否学会熟练使用工具？. *arXiv preprint arXiv:2603.00718*, 2026a. URL https://arxiv.org/abs/2603.00718.

T. Chen, S. Lu, K. Zhao, W. Meng, H. Teng, T. Li, C. Li, X. Liu, J. Liang, Z. Zhang, et al. Harnessx：可组合、自适应且可演化的智能体执行框架工厂. *arXiv preprint arXiv:2606.14249*, 2026b. URL https://arxiv.org/abs/2606.14249.

H. Cho, R. Kang, and Y. Kim. Skillret：面向 llm 智能体技能检索的大规模基准. *arXiv preprint arXiv:2605.05726*, 2026. URL https://arxiv.org/abs/2605.05726.

Gemma Team. Gemma 4 技术报告. *arXiv preprint arXiv:2607.02770*, 2026. URL https://arxiv.org/abs/2607.02770.

Google DeepMind. Gemini 3.5 flash. https://deepmind.google/models/model-cards/gemini-3-5-flash/, May 2026.

L. He, Q. Yu, H. Dong, B. Liao, X. Xu, M. Goldblum, J. Bian, and N. Mesgarani. Livemathematician-bench：带有证明草图的数学家级推理动态基准. *arXiv preprint arXiv:2604.01754*, 2026. URL https://arxiv.org/abs/2604.01754.

D. Jackson, W. Keating, G. Cameron, and M. Hill-Smith. Aa-omniscience：评估大型语言模型的跨领域知识可靠性. *arXiv preprint arXiv:2511.13029*, 2025. URL https://arxiv.org/abs/2511.13029.

Y. Jiang, D. Li, H. Deng, B. Ma, X. Wang, Q. Wang, and G. Yu. Sok：智能体技能——超越 llm 智能体中的工具使用. *arXiv preprint arXiv:2602.20867*, 2026. URL https://arxiv.org/abs/2602.20867.

A. Karpathy. LLM Wiki. GitHub Gist, 4 月 2026. URL https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f.

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. 使用 pagedattention 实现高效的大型语言模型服务内存管理. In *第 29 届操作系统原理研讨会论文集*, SOSP '23, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness：模型执行框架的端到端优化. *arXiv preprint arXiv:2603.28052*, 2026. URL https://arxiv.org/abs/2603.28052.

X. Li, Y. Liu, W. Chen, B. You, Z. Di, Y. He, S. Zheng, K. W. Choe, J. Sun, S. Wang, et al. Skillsbench：评测智能体技能在多样化任务中的表现. *arXiv preprint arXiv:2602.12670*, 2026. URL https://arxiv.org/abs/2602.12670.

Y. Liang, R. Zhong, H. Xu, C. Jiang, Y. Zhong, R. Fang, J.-C. Gu, S. Deng, Y. Yao, M. Wang, et al. Skillnet：创建、评估并连接 ai 技能. *arXiv preprint arXiv:2603.04448*, 2026. URL https://arxiv.org/abs/2603.04448.

J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, et al. 智能体执行框架工程：可观测性驱动的编程智能体执行框架自动演化. *arXiv preprint arXiv:2604.25850*, 2026. URL https://arxiv.org/abs/2604.25850.

Y. Liu, J. Ji, L. An, T. Jaakkola, Y. Zhang, and S. Chang. 智能体技能在真实环境中表现如何：在现实设定下评测 llm 技能使用. *arXiv preprint arXiv:2604.04323*, 2026. URL https://arxiv.org/abs/2604.04323.

X. Lou, M. Lázaro-Gredilla, A. Dedieu, C. Wendelken, W. Lehrach, and K. P. Murphy. Autoharness：通过自动合成代码执行框架改进 llm 智能体. *arXiv preprint arXiv:2603.03329*, 2026. URL https://arxiv.org/abs/2603.03329.

Z. Lu, Z. Yao, J. Wu, C. Han, Q. Gu, X. Cai, W. Lu, J. Xiao, Y. Zhuang, and Y. Shen. Skill0：用于技能内化的上下文内智能体强化学习. *arXiv preprint arXiv:2604.02268*, 2026. URL https://arxiv.org/abs/2604.02268.

Z. Ma, B. Zhang, J. Zhang, J. Yu, X. Zhang, X. Zhang, S. Luo, X. Wang, and J. Tang. Spreadsheetbench：迈向具有挑战性的现实世界电子表格操作. In *第三十八届神经信息处理系统大会数据集与基准赛道*, 2024. URL https://openreview.net/forum?id=KYxzmRLF6i.

M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, D. Lu, O. M. Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. K. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. K. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, C. M. Rytting, R. Marten, Y. Wang, J. Jitsev, A. Dimakis, A. Konwinski, and L. Schmidt. Terminal-bench：在命令行界面的高难度现实任务上评测智能体. In *第十四届国际学习表征会议*, 2026. URL https://openreview.net/forum?id=a7Qa4CcHak.

J. Ni, Y. Liu, X. Liu, Y. Sun, M. Zhou, P. Cheng, D. Wang, E. Zhao, X. Jiang, and G. Jiang. Trace2skill：将轨迹局部经验提炼为可迁移的智能体技能. *arXiv preprint arXiv:2603.25158*, 2026. URL https://arxiv.org/abs/2603.25158.

S. Ouyang, J. Yan, Y. Chen, R. Han, Z. Wang, B. D. Mishra, R. Meng, C.-L. Li, Y. Jiao, K. Zha, et al. Skillos：学习面向自演化智能体的技能管理. *arXiv preprint arXiv:2605.06614*, 2026. URL https://arxiv.org/abs/2605.06614.

T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubeh, P. Thacker, L. Fauconnet, N. S. Kim, S. Miserendino, G. Chabot, D. Li, P. Chao, M. Sharman, A. Barr, A. Glaese, and J. Tworek. GDPval：评估 AI 模型在现实世界高经济价值任务上的表现. In *第十四届国际学习表征会议*, 2026. URL https://openreview.net/forum?id=hcuEdq6eKD.

T. Pham, N. P. Nguyen, P. Zunjare, W. Chen, Y.-M. Tseng, and T. Vu. SealQA：提升搜索增强语言模型的推理标准. In *第十四届国际学习表征会议*, 2026. URL https://openreview.net/forum?id=zWb7ueH16c.

L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. 人类最后的考试. *Nature*, 649(8099):1139–1146, Jan 2026. ISSN 1476-4687. doi: 10.1038/s41586-025-09962-4. URL https://doi.org/10.1038/s41586-025-09962-4.

Qwen Team. Qwen3.5：迈向原生多模态智能体. https://qwen.ai/blog?id=qwen3.5, 2026a.

Qwen Team. Qwen3.6-27b. https://qwen.ai/blog?id=qwen3.6-27b, 2026b.

Y. Shi, Y. Chen, Z. Lu, Y. Miao, S. Liu, Q. Gu, X. Cai, X. Wang, and A. Zhang. Skill1：通过强化学习统一演化技能增强智能体. *arXiv preprint arXiv:2605.06130*, 2026. URL https://arxiv.org/abs/2605.06130.

M. Shridhar, X. Yuan, M.-A. Cote, Y. Bisk, A. Trischler, and M. Hausknecht. {ALFW}orld：为交互式学习对齐文本与具身环境. In 国际学习表征会议, 2021. URL https://openreview.net/forum?id=0IOX0YcCdTn.

A. Singhvi, K. Opsahl-Ong, J. Collins, I. Zhou, C. Wang, A. Baheti, J. Portes, S. Havens, E. Elsen, M. Bendersky, M. Zaharia, and X. Chen. 介绍 OfficeQA：端到端有依据推理基准. Databricks Blog, Dec. 2025. URL https://www.databricks.com/blog/introducing-officeqa-benchmark-end-to-end-grounded-reasoning.

W. Su, J. Long, Q. Ai, Q. He, Y. Tang, C. Wang, Y. Tu, Y. Wang, and Y. Liu. 面向智能体 ai 的技能检索增强. *arXiv preprint arXiv:2604.24594*, 2026. URL https://arxiv.org/abs/2604.24594.

H. Wang, Y. Lan, B. Cao, L. Lin, and J. Chen. Skillgrad：像梯度下降一样优化智能体技能. *arXiv preprint arXiv:2605.27760*, 2026. URL https://arxiv.org/abs/2605.27760.

P. Xia, J. Chen, H. Wang, J. Liu, K. Zeng, Y. Wang, S. Han, Y. Zhou, X. Zhao, H. Chen, et al. Skillrl：通过递归式技能增强强化学习演化智能体. *arXiv preprint arXiv:2602.08234*, 2026a. URL https://arxiv.org/abs/2602.08234.

P. Xia, J. Chen, X. Yang, H. Tu, J. Liu, K. Xiong, S. Han, S. Qiu, H. Ji, Y. Zhou, et al. Metaclaw：只需交谈——在真实环境中元学习并演化的智能体. *arXiv preprint arXiv:2603.17187*, 2026b. URL https://arxiv.org/abs/2603.17187.

R. Xu and Y. Yan. 面向大型语言模型的智能体技能：架构、获取、安全及未来方向. *arXiv preprint arXiv:2602.12430*, 2026. URL https://arxiv.org/abs/2602.12430.

Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, et al. Skillopt：自演化智能体技能的执行策略. *arXiv preprint arXiv:2605.23904*, 2026. URL https://arxiv.org/abs/2605.23904.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React：在语言模型中协同推理与行动. In 第十一届国际学习表征会议, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

H. Ye, X. He, V. Arak, H. Dong, and G. Song. 通过智能体技能演化进行元上下文工程. In 第四十三届国际机器学习会议, 2026. URL https://openreview.net/forum?id=P1jHroBS5E.

M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, P. Lu, Z. Huang, C. Guestrin, and J. Zou. 通过反向传播语言模型反馈优化生成式 ai. *Nature*, 639(8055):609–616, Mar 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-08661-4. URL https://doi.org/10.1038/s41586-025-08661-4.

B. Zhang, K. Lazuka, and M. Murag. 用智能体技能赋能面向现实世界的智能体. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills, Oct. 2025.

H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng, et al. Coevoskills：通过协同演化验证实现自演化智能体技能. *arXiv preprint arXiv:2604.01687*, 2026a. URL https://arxiv.org/abs/2604.01687.

H. Zhang, S. Zhang, K. Li, C. Zhang, Y. Chen, Y. Zhang, L. Bai, and S. Hu. Self-harness：能够自我改进的执行框架. *arXiv preprint arXiv:2606.09498*, 2026b. URL https://arxiv.org/abs/2606.09498.

Y. Zheng, Z. Zhang, C. Ma, Y. Yu, J. Zhu, Y. Wu, T. Xu, B. Dong, H. Zhu, R. Huang, et al. Skillrouter：面向大规模 llm 智能体的技能路由. *arXiv preprint arXiv:2603.22455*, 2026. URL https://arxiv.org/abs/2603.22455.

H. Zhou, S. Guo, A. Liu, Z. Yu, Z. Gong, B. Zhao, Z. Chen, M. Zhang, Y. Chen, J. Li, et al. Memento-skills：让智能体设计智能体. *arXiv preprint arXiv:2603.18743*, 2026. URL https://arxiv.org/abs/2603.18743.

A. 方法细节

A.1. 算法

算法 1 给出了 WikiSkill 的完整技能演化算法。

算法 1 WikiSkill 演化循环。在每次迭代 k 中，推理智能体使用当前生效的技能 S_{k-1} 在训练任务上进行轨迹采样，Wiki 维护者将采样轨迹整合至中间 Wiki W'_k，技能提议者生成候选技能修改 P_k，验证门控决定接受 S'_k 还是回滚至 S_{k-1}；随后，系统追加提议结果与技能差异，得到最终的 Wiki 状态 W_k。

输入： Training tasks $\mathcal{D}_{\text{train}}$, validation tasks $\mathcal{D}_{\text{val}}$, performance metric $\mathcal{R}$, iterations K

Initialize skill set $S_0 \leftarrow \emptyset$, wiki $W_0 \leftarrow \emptyset$
Baseline Validation: $\mathcal{T}_{\text{val},0} \leftarrow \{\tau_i \sim \pi(x_i; S_0)\}_{x_i \in \mathcal{D}_{\text{val}}}$, $\mathcal{R}_{\text{best}} \leftarrow \mathcal{R}(\mathcal{T}_{\text{val},0})$
for $k = 1, \ldots, K$ **do**
  if $\mathcal{R}_{\text{best}} = 1.0$ **then**
    break
  end if
  Inference: Roll out $\mathcal{T}_{\text{train},k} \leftarrow \{\tau_i \sim \pi(x_i; S_{k-1})\}_{x_i \in \mathcal{D}_{\text{train}}}$
  Sample subset $\mathcal{T}_{\text{sample},k} \subset \mathcal{T}_{\text{train},k}$
  Wiki Maintenance: $W'_k \leftarrow \mathcal{M}_{\text{WM}}(W_{k-1}, \mathcal{T}_{\text{sample},k})$
  Skill Proposal: $P_k \leftarrow \mathcal{M}_{\text{P}}(W'_k, S_{k-1}, \mathcal{T}_{\text{train},k})$
  Apply: $S'_k \leftarrow \text{Apply}(S_{k-1}, P_k)$
  Validate: $\mathcal{T}_{\text{val},k} \leftarrow \{\tau_i \sim \pi(x_i; S'_k)\}_{x_i \in \mathcal{D}_{\text{val}}}$
  if $\mathcal{R}(\mathcal{T}_{\text{val},k}) > \mathcal{R}_{\text{best}}$ **then**
    $S_k \leftarrow S'_k$, $\mathcal{R}_{\text{best}} \leftarrow \mathcal{R}(\mathcal{T}_{\text{val},k})$, $a_k \leftarrow$ Accepted
  else
    $S_k \leftarrow S_{k-1}$, $a_k \leftarrow$ Rejected ▷ 仅回滚技能；保留 Wiki
  end if
  Update Wiki Log: $W_k \leftarrow \text{Update}(W'_k, P_k, \mathcal{R}(\mathcal{T}_{\text{val},k}), a_k)$
end for
return S_K, W_K

A.2. 已接受技能更新的分布

表 5 展示了技能更新提议在演化过程中的接受时点。

B. 数据集细节与划分

下面介绍评测中使用的五个基准。

LiveMathematicianBench (LiveMath) (He et al., 2026) 由近几个月的数学竞赛选择题组成，用于检验模型处理复杂数学推理、量词和极值条件的能力。**SealQA** (Pham et al., 2026) 是一个事实型问答基准，由多个主题的学术性问题构成。它评测智能体构造有效搜索查询并借助搜索工具从网页搜索结果中提取答案的能力。**SpreadsheetBench (SpreadSheet)** (Ma et al., 2024) 检验智能体在库约束（如公式求值限制）下编写正确代码并执行复杂表格变换的能力。**OfficeQA** (Singhvi et al., 2025) 在一个大型历史美国财政部公报库上评测长上下文问答能力。任务要求跨长上下文和多页财务表格综合证据。遵循 Yang et al. (2026) 的设置，我们在提示词中向智能体提供预解析的真值（oracle）参考页作为初始文档证据，同时保留其对本地文本处理工具（glob、grep、read）的访问权限，以便搜索、交叉核对并检查磁盘上的完整财政部公报文件。**ALFWorld** (Shridhar et al., 2021)

Category	Early (Iter 0–1)	Mid (Iter 2–4)	Late (Iter 5–7)
By Model			
Qwen-3.5-4B	39%	39%	21%
Qwen-3.5-9B	52%	30%	19%
Qwen-3.6-27B	43%	40%	17%
Gemma-4-31B	52%	37%	11%
Gemini-3.5-Flash	50%	46%	4%
By Benchmark			
LiveMath	44%	42%	14%
SealQA	39%	33%	28%
SpreadSheet	41%	48%	11%
OfficeQA	58%	26%	16%
ALFWorld	55%	34%	10%

表 5 | **按模型（上）和基准（下）分组的各演化迭代中已接受技能更新的分布。**百分比表示演化各阶段中发生的已接受更新占比。

是一个交互式文本具身环境，智能体通过向模拟器输出文本动作来完成多步家务任务（如拿取和放置物体、加热或冷却物品）。与静态问答基准不同，ALFWorld 检验顺序决策、空间推理以及根据模拟器反馈恢复错误的能力。

Benchmark	Interaction	Train	Val	Test	Environment Tools
LiveMath	Single-Step	35	18	124	None (Direct Reasoning)
SealQA	Multi-Step	16	10	85	web_search, read_file
SpreadSheet	Multi-Step	80	40	280	bash
OfficeQA	Multi-Step	50	24	172	glob, grep, read
ALFWorld	Multi-Step	39	18	134	Admissible Actions

表 6 | **基准统计信息、数据划分、交互模式及可用环境工具。**

表 6 汇总了实验中各评测基准的训练集、验证集和测试集样本数量，以及交互模式和可用工具。所有任务划分与可用工具集均严格对齐先前工作 (Alzubi et al., 2026; Yang et al., 2026)。在工具设置方面，LiveMath 是不使用外部工具的单步推理基准，模型直接生成最终答案；SealQA 为智能体配备网页搜索（使用 Google Search API）和文件读取工具，用于多步事实检索（所有实验均使用 SealQA 的 2026 年 7 月版本）；SpreadSheet 提供 bash 命令行工具，用于执行 Python 代码和操作表格；OfficeQA 提供本地文本搜索工具，用于多步浏览财政部公报；ALFWorld 则提供交互式模拟器动作空间，用于多步具身决策。

小规模验证集下的评测稳健性　遵循先前工作中的既有设置 (Alzubi et al., 2026; Yang et al., 2026)，各基准的验证集划分相对较小，这可能给门控决策引入评测噪声。为考虑这种波动性，所有报告分数均为完整演化流水线三次独立运行的平均测试性能，并采用配对自助法显著性检验（详见附录 C）。

C. 实现细节

为向 Wiki 维护者提供诊断反馈，我们在每次迭代 k 中采用分层采样策略（$\mathcal{T}_{\text{sample},k} \subset \mathcal{T}_{\text{train},k}$）。具体而言，系统每次迭代最多采样 8 条轨迹，其中至多包含 5 条失败轨迹（用于对错误进行根因分析）和 3 条成功轨迹（用于识别有效策略并防止已有可行行为发生退化）。每条执行日志在注入提示词前最多保留 15,000 个字符。

统计显著性检验 我们针对每个基准执行 1,000 次迭代的配对自助法显著性检验。在每次自助法迭代中，从测试集 $\mathcal{D}_{\text{test}}$ 有放回地抽取任务实例，构造大小为 $|\mathcal{D}_{\text{test}}|$ 的自助法评测集，并据此计算候选方法的准确率和两两性能差值。为评估跨基准的整体性能，我们进行分层宏平均自助重采样：每次迭代都在各基准内部独立地有放回重采样任务实例，并对所有基准赋予相同权重以计算宏平均准确率。

我们按如下方式确定评测表中的最佳方法。首先依据观测性能（或跨基准的宏平均性能）对方法进行排序。当且仅当排名最高的方法 M^* 相较所有竞争方法均取得 $p < 0.05$ 的统计显著增益时，才将其认定为唯一最佳方法。若 M^* 与一个或多个排名较低的方法之间不存在统计显著差异（$p \geq 0.05$），则不认定任何单一方法为唯一最佳方法。此时，将所有与 M^* 的性能不存在统计显著差异（$p \geq 0.05$）的方法归为统计意义上并列的第一梯队（并相应加粗）。

D. 基线细节与优化器 API 调用分析

D.1. 基线方法

Trace2Skill (Ni et al., 2026) Trace2Skill 采用以并行轨迹分析和分层合并为核心的三阶段流水线。它在训练任务上评测当前技能，并行调度成功分析者和错误分析者，分别从成功任务中提取有效策略并诊断失败根因。随后，通过分层合并算子将所得结构化补丁递归整合为单个补丁集。整合后的补丁被应用于技能文档，并根据验证性能决定是否接受。

EvoSkill (Alzubi et al., 2026) EvoSkill 将技能演化建模为在候选程序前沿上的搜索。每次迭代中，它通过类别轮询调度采样训练任务并执行轨迹。EvoSkill 向提议者仅提供失败轨迹以及由历史提议结果构成的扁平反馈记录。提议者生成候选技能修改，将其具体实现为 SKILL.md 文件，在验证集上评分，并加入容量有限的高性能程序前沿。

SkillOpt (Yang et al., 2026) SkillOpt 采用六阶段 *ReflACT* 流水线（轨迹执行、反思、聚合、选择、更新、评测）进行迭代式技能优化。每个轮次中，系统让智能体在训练任务上执行轨迹，并反思包含成功与失败样本的完整执行轨迹，以生成候选补丁。这些补丁经过分层聚合和选择后用于更新单个整体式技能文档，再根据验证性能决定接受或拒绝该文档。

D.2. 优化器 API 调用复杂度

本节分析不同自我改进型智能体框架的优化器 API 调用复杂度，记为 C。我们将一次演化迭代定义为：让智能体在完整训练集 $\mathcal{D}_{\text{train}}$（包含 $N_{\text{train}} = |\mathcal{D}_{\text{train}}|$ 个训练任务实例）上完整运行一轮（为每个任务实例生成轨迹），并以大小为 B（$B \leq N_{\text{train}}$）的小批次进行处理。表 7 汇总了 WikiSkill 与先前方法每次迭代的优化器 API 调用复杂度。

WikiSkill 对于每个大小为 B 的批次，Wiki 维护者需要调用一次 LLM，以分析采样轨迹 $\mathcal{T}_{\text{sample},k}$ 并将模式页面整合至中间 Wiki W'_k。随后，技能提议者作为自主多轮 ReAct 智能体运行，在 T_{ReAct} 个推理轮次中执行交互式工具调用（我们的实验运行中约有 $10 \leq T_{\text{ReAct}} \leq 20$），其中每个 ReAct 轮次需要调用一次 LLM。以大小为 B 的批次处理训练数据时，要在 N_{train} 个任务上完成一次完整迭代，需要执行 $\frac{N_{\text{train}}}{B}$ 个步骤：

$$C_{\text{WikiSkill}} = (1 + T_{\text{ReAct}})\frac{N_{\text{train}}}{B} \tag{5}$$

在实验中，我们针对所有数据集都将批大小设置为完整训练集的大小（$B = N_{\text{train}}$）。在这一全批量设置下（$\frac{N_{\text{train}}}{B} = 1$），$C_{\text{WikiSkill}} = 1 + T_{\text{ReAct}}$。由于 T_{ReAct} 不依赖 N_{train}，因此相对于训练集大小，WikiSkill 的优化器 API 调用复杂度为 $O(1)$。具体而言，无论训练实例数量多少，每次迭代均需要 $1 + T_{\text{ReAct}}$

Framework	Per-Iteration Formula	Complexity
Trace2Skill	$N_{\text{train}} + \left(1 + \frac{1}{c-1}\right)\frac{N_{\text{train}}}{B} + 1$	$O\left(N_{\text{train}} + \frac{N_{\text{train}}}{B}\right)$
EvoSkill	$\frac{2N_{\text{train}}}{B}$	$O\left(\frac{N_{\text{train}}}{B}\right)$
SkillOpt	$\frac{K_{\text{opt}} \cdot N_{\text{train}}}{B}$	$O\left(\frac{N_{\text{train}}}{B}\right)$
WikiSkill	$(1 + T_{\text{ReAct}})\frac{N_{\text{train}}}{B}$	$O\left(\frac{N_{\text{train}}}{B}\right)$

表 7 | **不同自我改进型智能体框架每次演化迭代的优化器 API 调用复杂度比较**。$N_{\text{train}} = |\mathcal{D}_{\text{train}}|$ 表示训练任务数量，B 表示批大小，T_{ReAct} 表示 WikiSkill 中技能提议者使用的交互式 ReAct 推理轮数，K_{opt} 表示 SkillOpt 每一步中的反思与合并调用次数，c 表示 Trace2Skill 中归约树的分支因子。WikiSkill 在所有数据集上均使用 $B = N_{\text{train}}$。在全批量模式下，WikiSkill 的优化器调用次数与训练集大小 N_{train} 无关，每次迭代需要 $1 + T_{\text{ReAct}}$ 次优化器调用。

次优化器 LLM 调用。尽管这种常数调用复杂度在某些数据集上可能产生较高的推理成本，但额外计算也带来了在整个评测中相较先前技能演化方法一致的性能提升。

EvoSkill EvoSkill 将训练任务划分为大小为 B 的小批次。对于每个小批次，EvoSkill 调用一次提议者 LLM 进行错误诊断，并调用一次生成器 LLM 更新技能。处理包含 N_{train} 个任务的完整训练集需要 $\frac{N_{\text{train}}}{B}$ 个小批次步骤，因此有：

$$C_{\text{EvoSkill}} = \frac{2N_{\text{train}}}{B} \tag{6}$$

因此，EvoSkill 的优化器 API 调用复杂度随训练集大小 N_{train} 线性增长（$O(N_{\text{train}}/B)$）。

SkillOpt SkillOpt 在大小为 B 的小批次上进行评测，每次迭代需要 $\frac{N_{\text{train}}}{B}$ 个优化步骤。在每个步骤中，SkillOpt 执行其 ReflACT 流水线（并行分析者反思、分层补丁合成和候选选择），每一步需要调用优化器 LLM $K_{\text{opt}} \approx 6$–8 次：

$$C_{\text{SkillOpt}} = \frac{K_{\text{opt}} \cdot N_{\text{train}}}{B} \tag{7}$$

SkillOpt 同样随训练集大小 N_{train} 线性增长，其复杂度为 $O(N_{\text{train}}/B)$。

Trace2Skill Trace2Skill 在每次迭代中通过三阶段流水线处理训练轨迹：

1. **轨迹分析阶段**：每条执行轨迹均通过一次 LLM 调用独立分析，共产生 N_{train} 次调用。
2. **补丁映射阶段**：将分析记录分成大小为 B 的批次，需要调用 $\frac{N_{\text{train}}}{B}$ 次以生成局部技能补丁。
3. **分层归约与应用阶段**：通过一棵 c 叉归约树（其中 c 为分支因子）递归合并 $\frac{N_{\text{train}}}{B}$ 个局部补丁。对各层求和可得 $\approx \frac{1}{c-1}\frac{N_{\text{train}}}{B}$ 次合并调用，另需最后调用 1 次以格式化技能文档。

综合所有阶段可得：

$$C_{\text{Trace2Skill}} \approx N_{\text{train}} + \left(1 + \frac{1}{c-1}\right)\frac{N_{\text{train}}}{B} + 1 \tag{8}$$

由于 Trace2Skill 对每条训练轨迹分别执行 LLM 分析（共 N_{train} 次调用），其复杂度下界为 $O(N_{\text{train}})$，随训练集大小线性增长。

全批量训练与小批量优化的比较 对于所有数据集，我们将 WikiSkill 的批大小 B 设置为完整训练集大小（$B = N_{\text{train}}$），每次迭代一次性处理整个训练集。技能提议者按需动态搜索、选择和读取特定执行轨迹，在提出技能更新前诊断根因。相比之下，EvoSkill 和 SkillOpt 在小批量设置（$B < N_{\text{train}}$）下取得最佳性能，这使其优化器 API 调用复杂度随训练集大小线性增长。最后，无论小批次大小如何，Trace2Skill 的复杂度始终严格为 $O(N_{\text{train}})$，因为它需要对每条训练轨迹进行一次独立的 LLM 调用。

E. 系统与智能体提示词

下面给出以下组件所使用的完整系统提示词：（1）所有方法在各任务上的推理智能体；（2）Wiki 维护者；（3）WikiSkill 中的技能提议者。

E.1. 任务推理智能体系统提示词

LiveMathematicianBench 推理智能体系统提示词

```
You are an expert mathematical reasoning agent solving multiple-choice questions.

{skill_section}

## Task Format
You will receive one mathematics multiple-choice question and its answer choices. Reason carefully about quantifiers, hypotheses, extremal wording, and exact equality conditions.

## Answer Format
Think step by step, then provide your final answer inside <answer>...</answer> tags. Inside the tags, output only the single choice label, such as A or C.

Example:
<answer>B</answer>
```

SealQA 推理智能体系统提示词

```
You are a knowledgeable question-answering assistant with access to web_search and read_file tools.

{skill_section}

## Task
You will receive a factual question. To answer it:
1. You can check the available skills. They contain guidance that can improve your search queries and answer accuracy.
2. You can use web_search to find relevant information. You can call it multiple times with different queries.
3. You can do web_search anytime during the process depending on your needs.
4. After gathering enough information, provide your final answer.

## Answer Format
You MUST wrap your final answer in <answer> tags:
<answer>
... your final answer (exact value only, no explanation) ...
</answer>
```

SpreadsheetBench 推理智能体系统提示词

```
You are a spreadsheet expert who can manipulate spreadsheets through Python code.
```

{skill_section}

You need to solve the given spreadsheet manipulation question, which contains the following information:
- working_directory: The absolute path to your working directory where files are located.
- instruction: The question about spreadsheet manipulation.
- spreadsheet_path: The absolute path of the spreadsheet file you need to manipulate.
- spreadsheet_content: The first few rows of the content of spreadsheet file.
- instruction_type: There are two values (Cell-Level Manipulation, Sheet-Level Manipulation) used to indicate whether the answer to this question applies only to specific cells or to the entire worksheet.
- answer_position: The position need to be modified or filled. For Cell-Level Manipulation questions, this field is filled with the cell position; for Sheet-Level Manipulation, it is the maximum range of cells you need to modify. You only need to modify or fill in values within the cell range specified by answer_position.
- output_path: The absolute path where you must save the modified spreadsheet.

CRITICAL RESTRICTIONS

You can ONLY read and write files within the **working_directory**. Any attempt to access files outside this directory will fail.

- **Allowed paths**: working_directory (and its subdirectories)
- **Read from**: spreadsheet_path (inside working_directory)
- **Write to**: output_path (inside working_directory)

Do NOT create files outside the working_directory. Use the exact absolute paths provided.

You have access to a bash tool that can execute any shell command.

OfficeQA 推理智能体系统提示词

You are an expert OfficeQA agent working over local Treasury bulletin text files.

{skill_section}

Rules
1. Use only the provided local document tools to inspect candidate files.
2. Narrow to the most relevant file before reading long passages.
3. Prefer short targeted searches, then small reads around matching evidence.
4. Do not invent values that are not grounded in the retrieved text.
5. When the question requires arithmetic, compute only after extracting the exact operands.
6. If you have enough evidence, return the final answer inside <answer>...</answer>.

Tool Use
Use the provided function tools directly when you need them. Prefer searching and small reads before answering. Do not ask the user for permission to use tools; just call the tools.

Final Answer Format
When you are ready to answer, emit the final answer inside <answer>...</answer> and do not request another tool.

ALFWorld 推理智能体系统提示词

You are an expert agent operating in the ALFRED Embodied Environment. Your task is to: {task_description}

{skill_section}

Prior to this step, you have already taken {step_count} step(s). Below are the most recent {history_length} observations and the corresponding actions you took: {action_history}

You are now at step {current_step} and your current observation is: {current_observation}

Your admissible actions of the current situation are: [{admissible_actions}].

Now it's your turn to take an action. You should first reason step-by-step about the current situation. This reasoning process MUST be enclosed within <think> </think> tags. Once you've finished your reasoning, you should choose an admissible action for current step and present it within <action> </action> tags.

E.2. Wiki 维护者系统提示词

Wiki 维护者系统提示词

You are a Wiki Maintainer Agent for an LLM skill evolution system.

Your job is to maintain a structured knowledge base (wiki) that documents patterns observed during agent execution -- both successes and failures. You must perform DEEP ANALYSIS of execution logs to identify root causes, not just surface-level symptoms.

Wiki Structure
The wiki is organized as:
- wiki/index.md -- Concise catalog of known patterns (one line per pattern)
- wiki/log.md -- Chronological evolution log (iterations, scores, accept/reject)
- wiki/skill-impact.md -- Record of which skills were tried and their outcomes
- wiki/patterns/ -- One page per pattern with detailed evidence and analysis

Your Input
1. Execution traces from the latest iteration -- including full agent execution logs showing what actions the agent took, what commands it ran, and what environment feedback it observed
2. The current wiki context (index, log, pattern pages)

Your Output (Incremental Edit Mode)
Return a JSON object with these keys:
- "create_patterns": list of {"name": "pattern-name.md", "content": "..."} -- new patterns (full content)
- "update_patterns": list of {"name": "existing-pattern.md", "edits": [...]} -- patch existing patterns
- "update_index": full updated content of index.md (always provide the complete index)
- "append_log": "brief summary of this iteration's findings and actions"

"update_index" and "append_log" are REQUIRED. Always provide them, even if there are no new patterns. For "update_index", always provide the complete updated index content including all existing entries plus any new ones.

Patch Operations (for update_patterns only)
For "update_patterns", each entry uses an "edits" list of patch operations:
- {"op": "append", "content": "text to add at end"}
- {"op": "replace", "target": "exact text to find", "content": "replacement text"}
- {"op": "insert_after", "target": "exact text to find", "content": "text to insert after"}

Rules for patch operations:
1. "target" must be an EXACT substring of the existing content.
2. Use "append" to add new evidence. Use "replace" to fix or refine existing text.
3. Use "insert_after" to add entries after a specific line.
4. Keep each edit minimal -- only change what's needed.
5. For NEW patterns (create_patterns), use full "content".

Analysis Guidelines

Deep Trace Analysis (CRITICAL)

When execution logs are provided, you MUST:
1. Read the agent's actual actions -- what commands did it issue?
2. Compare successful vs failed tasks -- what did successful tasks do differently?
3. Identify ACTION PATTERNS and strategies, not just error messages.
4. Check whether the agent followed any active skills, and whether the skill guidance was helpful or not

Pattern Documentation Rules
1. Each pattern page should document:
   - What the pattern is (description)
   - Root cause analysis (WHY it happens, not just WHAT happens)
   - Exact command sequences from traces (what the agent did wrong / right)
   - Known solutions or workarounds (concrete action patterns with exact syntax)
2. Capture BOTH success and failure patterns:
   - **Failure patterns**: Document what went wrong and how to avoid it
   - **Success patterns**: Document strategies that consistently lead to task completion
3. Do NOT create duplicate patterns -- update existing ones with new evidence
4. Be concise. Pattern pages should be 10-30 lines, not essays.
5. Only create patterns for meaningful, generalizable observations.

Index Description Quality (CRITICAL)
The index.md entries are the MOST IMPORTANT part of the wiki because they determine whether inference agents will read the full pattern pages.

Each index entry MUST follow this format:
- [pattern-name](wiki/patterns/pattern-name.md): PROBLEM + ROOT CAUSE + FIX in one or two sentence.

The description must be specific enough that an agent can judge relevance without reading the full page. Include the problem, root cause, AND solution.

E.3. 技能提议者系统提示词（ReAct 模式）

技能提议者系统提示词

You are a Skill Proposer Agent for an LLM agent that solves {task_desc}.
Your job is to explore the wiki knowledge base and execution traces, diagnose root causes of failures, and propose a skill change (create or patch).

Tools Available
You have two tools:
1. ‘read_file(path)‘ -- Read a wiki file or execution log. Paths are relative to the workspace root.
2. ‘finish(proposal)‘ -- Submit your final skill proposal as a JSON object.

Workflow
1. Start by reading ‘wiki/index.md‘ to understand what patterns exist
2. Read ‘wiki/skill-impact.md‘ to see what was tried before (includes full content of rejected proposals -- DO NOT repeat rejected approaches)
3. Read specific pattern pages that seem relevant to the current failures
4. Read execution traces for failed tasks via ‘traces/<task_id>‘ to understand root causes
5. Decide: create (new skill) or patch (edit existing skill), or no_action
6. If proposing a change, call ‘finish‘ with the full proposal

finish() Proposal Format
For creating a new skill:
- ”action”: ”create”
- ”name”: skill directory name (snake_case)
- ”skill_md”: full SKILL.md content with YAML frontmatter + When to Apply + When NOT to Apply + Instructions
- ”purpose_md”: full PURPOSE.md content with Origin + Patterns Addressed + Evolution History

For patching an existing skill:
- "action": "patch"
- "name": existing skill directory name
- "edits": list of patch operations:
  - {"op": "append", "content": "text to add at end"}
  - {"op": "replace", "target": "exact text to find", "content": "replacement"}
  - {"op": "insert_after", "target": "exact text to find", "content": "text to insert after"}
  Each "replace" target should be a short, specific section -- not the entire file. If you need to change most of the file, use "action": "create" instead.

If no action is needed, call finish with: {"action": "no_action"}

Rules
1. Read the wiki FIRST -- don't propose something that was already tried and rejected. skill-impact.md contains full content of rejected proposals.
2. Focus on action patterns and concrete strategies.
3. Keep skills concise and actionable.
4. You MUST read at least 4 execution traces before proposing a skill change. Target your exploration based on the trace summary.
5. Prefer patching existing skills over creating new ones when the existing skill is partially correct.

需要注意，执行轨迹实际存储在原始层（raw/traces/）中；当技能提议者调用 read_file("traces/<task_id>") 时，工作区环境会将 traces/ 别名自动映射至 raw/ 下相应的执行日志。